

Faiez BEN AMAR

Ingénieur Full-Stack .NET / Angular

6 ans d'expérience • Nantes • Mobilité France entière

E-mail benamarfaiez@gmail.com
Téléphone +33 7 51 42 84 60
LinkedIn linkedin.com/in/faiez-ben-amar
GitHub github.com/benamarfaiez

PROFIL

Ingénieur logiciel Full-Stack .NET / Angular, spécialisé dans la conception d'architectures robustes (Clean Architecture, DDD, CQRS) et la modernisation d'applications, du legacy (.NET Framework, WCF) aux stacks actuelles.

Habitué à intervenir sur des domaines métier exigeants (courtage financier réglementé, gestion documentaire multi-versions, gestion des avenants d'assurance) en traduisant des règles business en solutions techniques fiables, de l'analyse du besoin jusqu'à la mise en production, en autonomie complète.

Orientée qualité de code et transmission des bonnes pratiques, je porte une attention particulière à la conception d'architectures pérennes plutôt qu'à l'empilement de technologies.

COMPÉTENCES TECHNIQUES

Architectures & Pratiques	Clean Architecture, DDD, Microservices, N-Layer, CQRS, TDD, Principes SOLID, Design Patterns
Écosystème Backend	C#, .NET (Core / 6 / 8), ASP.NET Core, EF Core, Dapper, Java, Python — Legacy : WCF, .NET Framework 3.5 / 4.8, IIS
API & Intégration	REST, SOAP, Webhooks, OpenAPI/Swagger, Axway, ApiDog, Postman — API tierces : DocuSign, RingCentral, Airship, SMTP
Frontend & UI	Angular (12/13/14), React 19, TypeScript, JavaScript, RxJS, Bootstrap, HTML5, CSS3/SCSS, Material UI, PDF.js, Chart.js
Bases de données & Cache	SQL Server, MySQL, PostgreSQL, MemoryCache, DistributedCache (Redis)
DevOps & CI/CD	Docker, GitHub Actions, GitLab CI, Azure DevOps, Nexus, Vercel
Tests & Qualité	xUnit, NUnit, Moq, FluentAssertions, AutoFixture, ArchUnitNET, FluentValidation, Jasmine, Karma, Playwright, Jest, ESLint
Analyse & Performance	BenchmarkDotNet, SonarQube, Semgrep
Outils & Documentation	Git/GitHub/GitLab/Bitbucket/SourceTree, Confluence, Visio, SharePoint, UML, Jira, Zeplin

COMPÉTENCES TRANSVERSALES

Compréhension métier	Capacité à s'immerger dans des domaines réglementés ou complexes (courtage financier, gestion documentaire, assurance) pour traduire des règles business en spécifications techniques exploitables
Gestion de projet	Coordination technique, planification des sprints et suivi des livrables (méthodologie Agile Scrum)
Ingénierie des exigences	Analyse et traduction des besoins utilisateurs en spécifications techniques et fonctionnelles claires
Conception de solutions	Modélisation d'architectures logicielles complexes axées sur la performance, la fiabilité et l'évolutivité
Qualité technique	Revue de code, diffusion des bonnes pratiques et alignement des standards de développement au sein de l'équipe

LANGUES

Français	Courant — pratique professionnelle quotidienne
Anglais	Pratique professionnelle (lu, écrit, échanges techniques)

DIPLÔMES ET FORMATIONS

2020	Diplôme National d'Ingénieur en Sciences de l'Informatique — École Nationale des Sciences de l'Informatique, Manouba, Tunisie
2017	Cycle préparatoire aux études d'ingénieur (Math/Physique) — École Préparatoire de la Faculté des Sciences de Sfax, Tunisie

RÉSUMÉ DES MISSIONS

Association France Handicap (APF)	Secteur associatif — Depuis janv. 2026 — Mécénat de compétences via Aubay
Henner	Secteur assurance — Nov. 2024 – Avr. 2026 — Ingénieur Logiciel .NET
Euro Information	Secteur bancaire — Sept. 2022 – Juin 2024 — Ingénieur Logiciel Full-Stack
Canaccord Genuity Groupe	Secteur financier — Juil. 2020 – Août 2022 — Ingénieur Logiciel Full-Stack

EXPÉRIENCES PROFESSIONNELLES

Association France Handicap (APF) — Ingénieur Logiciel • depuis Mai 2026 • Mécénat de compétences via Aubay

Projet : Actions Associatives

Contexte : Conception de bout en bout d'une application moderne de gestion et de tableaux de bord pour le suivi des actions associatives.

- **Enjeu métier** : donner à l'association une vision consolidée et fiable de ses actions terrain — jusque-là suivies dans des fichiers Google Sheets dispersés — pour objectiver le pilotage auprès des équipes et des financeurs.
- **Architecture & Conception** : initialisation du projet « from scratch » en Clean Architecture sous .NET 8 et React 19, avec isolation stricte de la logique métier (Use Cases), pattern CQRS, API REST, handlers de commandes/requêtes, moteurs de validation (Validators) pour sécuriser les flux entrants, et moteur de mapping de données dynamique.
- **Stockage & Intégration** : mise en place d'un système de persistance basé sur la synchronisation et la manipulation de données via Google Sheets API et Apps Script.
- **Développement Frontend** : dashboard interactif haute performance avec filtres croisés dynamiques pour le suivi des KPIs métiers et des formulaires multi-étapes complexes.
- **Qualité & Fiabilité** : rédaction des spécifications techniques et stratégie de tests automatisés (xUnit, Moq) combinant tests unitaires et d'intégration avec mock complet des API tierces.
- **DevOps & CI/CD** : intégration et déploiement continu via GitHub Actions pour automatiser la mise en production.

Environnement technique : C#, .NET 8, Clean Architecture, CQRS, FluentValidation, xUnit, Moq, React 19, TypeScript, Google Sheets API, Chart.js, GitHub Actions.

Projet : AssetFlow_Core — Projet R&D interne

Contexte : Application R&D de gestion d'actifs, de tickets de maintenance et d'assistance IA.

- **Enjeu métier** : fiabiliser l'astreinte technique en assignant automatiquement le bon expert selon la typologie et la criticité de l'incident, plutôt que de s'appuyer sur une répartition manuelle source d'erreurs et de retards.
- **Architecture & Design** : solution « from scratch » en Clean Architecture et Domain-Driven Design (DDD) sous .NET 8, avec isolation de la logique métier et découplage via le pattern CQRS.
- **Moteur d'assignation intelligent** : moteur de résolution dynamique d'équipes (pattern Strategy) automatisant l'attribution des astreintes selon la typologie et la criticité des incidents.
- **Intégration IA & RAG** : module d'IA basé sur une architecture RAG pour assister la résolution des tickets, avec couche d'anonymisation des flux garantissant la conformité RGPD avant envoi aux LLMs.

- **Performance & Qualité** : mise en cache avancée validée par BenchmarkDotNet, tests automatisés (xUnit, Moq), contrôle de la dette technique (SonarQube).
- **Temps réel & CI/CD** : notifications d'équipe via SignalR, conteneurisation Docker et pipelines GitHub Actions.

Environnement technique : C#, .NET 8, Clean Architecture, DDD, CQRS, Pattern Strategy, RAG, BenchmarkDotNet, SignalR, SonarQube, FluentValidation, xUnit, Moq, Docker, GitHub Actions.

Henner (Nantes) — *Ingénieur Logiciel .NET • Novembre 2024 – Avril 2026*

Projet : Push Notification

Contexte : Traitement automatisé (batch) sous .NET Framework 3.5 (contraintes legacy du client) dédié à la synchronisation de données et à l'envoi quotidien de notifications vers des applications mobiles via l'API Airship.

- **Enjeu métier** : une notification était considérée « non traitée » lorsque son statut en base n'avait pas été mis à jour dans un délai de 3 jours ; le batch devait résorber ce backlog de façon fiable, sans surcharger les utilisateurs finaux.
- **Solution technique** : traitement par lots d'utilisateurs plutôt que notification par notification pour éviter de bombarder l'utilisateur, avec routage dynamique par type de notification et mise à jour du statut en base une fois le lot traité avec succès.
- **Architecture & flux** : logique séquentielle du batch (récupération des assurés, routage dynamique) et pattern Repository pour isoler les accès aux données MySQL.
- **Génération de données** : moteur de production de fichiers CSV à forte volumétrie, avec gestion des canaux Singulier/Pluriel et intégration aux listes statiques Airship.
- **Intégration API** : client HTTP bas niveau (HttpRequest) vers l'API Airship, incluant la gestion des tokens d'authentification, des timeouts et des codes de retour.
- **Stratégie de test** : matrice de validation combinant les cas de flux (0, 1/N éléments Singuliers/Pluriels) sur les 3 types métiers (Msg, RM, Pj) ; debug des payloads d'API via Postman.
- **Déploiement** : cycle de mise en production (compilation en mode Release, packaging des scripts .bat et fichiers de configuration) et déploiement sur serveurs IIS.

Environnement technique : C#, .NET Framework 3.5, IIS, HttpRequest, MySQL, Airship API, Postman, GitLab.

Projet : RingCentral Webhook

Contexte : Conception d'un microservice .NET 6 exposant des API REST pour le traitement en temps réel des messages et pièces jointes issus de plusieurs instances RingCentral.

- **Enjeu métier** : les messages et pièces jointes récupérés depuis plusieurs instances RingCentral alimentaient directement d'autres projets internes en aval ; toute perte, doublon ou latence sur ce flux se propageait à toute la chaîne.
- **Solution technique** : parallélisation des appels asynchrones vers les différentes instances pour absorber le flux en temps réel, combinée à une gestion de l'idempotence garantissant qu'un même message n'était jamais traité ni transmis deux fois, même en cas de rejeu ou d'erreur transitoire.
- **API REST & intégration** : endpoints dédiés à la gestion des webhooks RingCentral (extraction des pièces jointes, récupération des détails et listage des messages par agent).
- **Multi-tenancy** : architecture multi-tenancy avec authentification isolée par instance pour garantir la stricte segmentation des accès aux données.
- **Sécurité** : validation des signatures des webhooks entrants et monitoring du trafic périmétrique via Axway API Gateway.
- **Qualité du code** : modélisation par DTOs (AutoMapper), validation des entrées (FluentValidation).
- **Tests** : suite xUnit avec isolation des dépendances (Moq) et assertions fluides (FluentAssertions), intégrée au pipeline GitLab CI avec contrôle de la dette technique via SonarQube.

Environnement technique : C#, .NET 6, ASP.NET Core, HttpClientFactory, AutoMapper, FluentValidation, xUnit, Moq, FluentAssertions, Axway, RingCentral API, Postman, GitLab CI.

Projet : Gestion des avenants (Courtier)

Contexte : Refonte complète du système de gestion des avenants et remplacement d'une solution externe obsolète par une Web API .NET 6 connectée à PostgreSQL, dans une architecture microservices.

- **Enjeu métier** : chaque type d'avenant (résiliation, changement de garantie, ajout de bénéficiaire...) obéit à des règles de traitement propres ; l'ancien outil externe ne permettait pas de faire évoluer ces règles sans dépendre de l'éditeur. Objectif : reprendre la main sur cette logique métier en interne, et pouvoir ajouter de nouveaux types d'avenants.
- **Extensibilité métier** : architecture extensible grâce au pattern Factory pour l'instanciation dynamique des types d'avenants, sans modification du code existant.
- **Solution technique — recherche & pagination** : repository Dapper combinant les opérations CRUD classiques avec une recherche multicritère (construction dynamique de requêtes SQL selon les filtres actifs), un tri configurable et une pagination gérée au niveau base de données pour rester performant sur de gros volumes.
- **Modélisation & migration** : schéma relationnel PostgreSQL et scripts de migration (montée/descente de schéma) via FluentMigrator.
- **Cohérence transactionnelle** : pattern Unit of Work dans les services métier pour garantir l'atomicité des opérations en base de données.
- **Qualité & CI/CD** : tests unitaires et d'intégration xUnit/Moq (fixtures gérant les cycles Up/Down de FluentMigrator pour isoler le schéma de test), pipeline CI/CD GitLab CI avec conteneurisation Docker et contrôle qualité SonarQube.

Environnement technique : C#, .NET 6, ASP.NET Core, PostgreSQL, Dapper, FluentMigrator, AutoMapper, xUnit, Moq, Docker, GitLab CI, SonarQube, Postman.

Projet : Migration .NET 6 → .NET 8

Contexte : Pilotage technique de la migration de plus de 100 projets interdépendants (Library, API, Services) et conception d'un outil d'automatisation sur mesure couvrant environ 80 % des projets sans intervention manuelle.

- **Enjeu** : migrer plus de 100 projets interdépendants vers .NET 8 sans interrompre les équipes qui en dépendent au quotidien, ni casser la compatibilité entre eux.
- **Analyse des dépendances** : cartographie des graphes de dépendances entre SDK et API ; stratégies de migration par groupes cohérents pour prévenir toute rupture de compatibilité.
- **Outil d'automatisation (LibGit2Sharp)** : outil C# pilotant le workflow complet — création de branches Git, mise à jour des Target Frameworks, montée de version des packages NuGet, mise à jour des Dockerfile et pipelines GitLab CI, exécution automatique du build et des tests.
- **Monitoring & reporting** : suivi en temps réel du statut de chaque composant (Build / Tests / Push) avec génération de rapports de conformité via ClosedXML.
- **Migration manuelle ciblée** : correction des incompatibilités résiduelles sur les ~20 % de projets restants — remplacement des méthodes obsolètes et résolution des erreurs de build non automatisables.
- **Publication par paliers** : déploiement ordonné sur Nexus (flux NuGet-Beta), garantissant l'absence de ruptures de dépendances transverses et la réutilisation immédiate des SDK migrés.

Environnement technique : C#, .NET 8, LibGit2Sharp, NuGet, Nexus, Docker, GitLab CI, ClosedXML, Serilog, PowerShell, xUnit.

Euro Information (Nantes) — Ingénieur Logiciel Full-Stack • Septembre 2022 – Juin 2024

Projet : Pixis

Contexte : Conception et développement fullstack d'une plateforme interne de consultation de documents multi-fichiers (PDF, vidéo, HTML...) avec gestion du versioning ; chaque document pouvant agréger plusieurs fichiers hétérogènes et conserver un historique complet de ses versions.

Équipe & méthode : Équipe de 4 personnes en Agile Scrum (suivi sous TFS). Collaboration quotidienne avec l'équipe design/ergonomie pour le respect des parcours utilisateur, et avec l'équipe de test (QA) pour le suivi de la recette et la validation fonctionnelle.

- **Enjeu métier — droits par rôle** : un document pouvait regrouper plusieurs fichiers et versions. J'ai modélisé deux niveaux de droits : le créateur/rédacteur du document, seul habilité à créer une nouvelle version ou à archiver une version existante, et l'utilisateur standard, limité au signalement d'anomalies — signalement qui déclenche automatiquement un e-mail au créateur pour garantir la traçabilité des corrections.
- **Modernisation de la couche services** : analyse de l'existant WCF, puis refonte des contrats de service (ServiceContracts / DataContracts) en API REST exposant une hypermédia HATEOAS/HAL pour piloter la gestion documentaire, l'historique de consultation et les favoris.
- **Performance** : cache applicatif partagé (MemoryCache) et pipelines d'interception (Message Inspectors) pour centraliser les traitements transverses ; transfert groupé de fichiers multi-formats via le mode Streaming de WCF.

- **Versioning documentaire** : gestion des versions successives de chaque document avec accès à toute version antérieure, et modélisation du schéma de données adapté à l'historisation.
- **Moteur de recherche** : recherche full-text intra-document côté frontend, avec surlignage dynamique des occurrences trouvées et affichage de leur nombre, pour guider l'utilisateur dans des documents volumineux.
- **Rendu documentaire** : parsing de documents HTML extraits de la base de données avec injection dynamique de styles CSS pour un rendu visuel uniforme entre sources et versions.
- **Frontend (Angular 12, from scratch)** : Architecture modulaire Core / Shared / Feature Modules avec Lazy Loading ; interface de consultation multi-format et multi-version; intégration des maquettes Zeplin en étroite collaboration avec l'équipe design.
- **Documentation, Qualité & Recette** : Rédaction de la documentation technique (architectures logicielles et spécifications des API) ; écriture des tests unitaires (Jasmine côté frontend). collaboration étroite avec l'équipe de test pour le cadrage des cas de recette, le suivi des anomalies et la garantie du niveau de qualité.

Environnement technique : C#, .NET Framework 4.8, WCF, Entity Framework, SQL Server, IIS, Angular 12, TypeScript, RxJS, Bootstrap, SCSS, PDF.js, Ngx-Videoangular, Jasmine, TFS, GitLab, Postman, Zeplin. Méthode : Agile Scrum.

Canaccord Genuity Groupe — Ingénieur Logiciel Full-Stack • Juillet 2020 – Août 2022

Projet : Directfolio

Contexte : Évolution d'une plateforme responsive de courtage à escompte, dans un cadre réglementé par l'autorité canadienne des courtiers, automatisant le parcours client depuis l'ouverture de compte jusqu'à la signature électronique via DocuSign, avec support multilingue.

Équipe & méthode : Équipe de 8 personnes (1 Product Owner, 1 Scrum Master, 2 testeurs QA, 4 développeurs) en Agile Scrum (sprints de 2 semaines sur Jira). Collaboration quotidienne et étroite avec le PO pour la clarification fonctionnelle, l'équipe Design (UI/UX) pour l'intégration visuelle, et la QA pour la validation des cas de recette.

- **Enjeu métier** : La réglementation impose de valider entièrement le dossier client (état civil, situation financière, choix du compte, comptes joints) avant d'autoriser la signature électronique. Identification d'un risque de non-conformité lié au contournement possible de cette règle.
- **Notifications e-mail** : service d'envoi SMTP couvrant chaque étape du parcours d'inscription, avec génération dynamique du contenu via des templates Razor (CSHTML).
- **Gestion documentaire** : service connecté à Azure Blob Storage V12 (authentication DefaultAzureCredential) pour le stockage, la récupération, l'upload, la suppression et la recherche sécurisés des documents d'onboarding.
- **Signature électronique (DocuSign)** : mise à jour dynamique des enveloppes (TextTabs, CheckboxTabs) via la technique de l'Anchor Tagging sur les documents PDF.
- **API & contrôleurs** : Conception et exposition des API REST de gestion documentaire, consommées côté frontend pour une interaction fluide entre les couches.
- **Frontend (Angular 13)** : Architecture en modules chargés à la demande (Lazy Loading), optimisation des performances d'affichage (stratégie OnPush, directives TrackBy), support multilingue via ngx-translate, et intégration fidèle des maquettes transmises par l'équipe design via Zeplin.
- **Documentation** : Rédaction de la documentation technique (architecture, spécifications d'API, guides d'exploitation), traduction des user stories avec le PO en spécifications techniques.
- **Qualité & déploiement** : refactoring des mappings AutoMapper, traduction des user stories en spécifications techniques, revues de code et pair programming ; déploiement et suivi en pré-production via Azure DevOps ; couverture de tests supérieure à 80 % (xUnit/Moq côté backend, Jasmine/Karma côté frontend).

Environnement technique : Backend : ASP.NET Core 3.1, C#, EF Core, SQL Server, SMTP, DocuSign. Frontend : Angular 13, TypeScript, RxJS, SCSS, Bootstrap. DevOps : Azure Blob Storage, Azure DevOps, Git (Bitbucket/GitHub). Tests : xUnit, Moq, Jasmine, Karma, Postman, Jira, Zeplin. Méthode : Agile Scrum.